דף סיכום בחינה

מזהה סטודנט: [redacted]

מזהה קורס: 20594 שם קורס: מערכות הפעלה

מספר שאלה	הערה	ניקוד מירבי	ציון
1	8+9	22.00	17.00
2		15.00	10.00
3		18.00	15.00
4		5.00	5.00
5		5.00	1.00
6		5.00	4.00
7		5.00	5.00
8		5.00	5.00
9		5.00	5.00
10		5.00	5.00
11		5.00	0.00
12		5.00	5.00

ציון בחינה סופי : 77.00

הבחינה הבדוקה בעמודים הבאים

[redacted]

מספר התלמיד הנבחן
רשום את כל תשע הספרות

[redacted]

האוניברסיטה הפתוחה

ט' באדר א' תשפ"ב

מס' שאלון - 521

10
בפברואר 2022

מס' מועד 86

סמסטר 2022א

20594 / 4

שאלון בחינת גמר

20594 - מערכות הפעלה

משך בחינה: 3 שעות

בשאלון זה **9** עמודים

מבנה הבחינה:

קראו בעיון לפני שתתחילו בפתרון הבחינה!

א. המבחן מורכב משלושה חלקים.

ב. בחלקים א ו - ב מופיעות שאלות פתוחות. ענו תשובות מלאות, בכתב קריא ובקיצור נמרץ. אין חובה להשתמש בכל השורות המוקצות לצורך התשובות, אך אין לחרוג מהמקום המוקצה.

ג. בחלק ג (שאלות אמריקאיות) עליכם לבחור בכל פעם בתשובה יחידה מבין התשובות המוצעות ולהקיף בעיגול את אות התשובה שבחרתם.

את התשובות לכל השאלות יש לכתוב בשאלון הבחינה.

חומר עזר:
כל חומר עזר אסור בשימוש, פרט למחשבון, שאינו אוצר מידע.

בהצלחה !!!

החזירו
למשגיח את השאלון
וכל עזר אחר שקיבלתם בתוך מחברת התשובות

521 שאלון 86.66.6 M1

חלק א (55 נקודות)

ענו על השאלות 1-3.

שאלה 1 (22 נקודות)

שני הסעיפים מתייחסים לבעיה הבאה.

במערכת ישנם שלושה תהליכונים (THREADS) A, B ו-C. יש לכתוב אלגוריתם (בפסאודוקוד או דמוי C) המממש קטע קריטי בעל התכונות הבאות:

אם A - נמצא בקטע הקריטי, אזי הוא נמצא בו לבדו.

תהליכונים B ו-C יכולים להימצא בקטע הקריטי בו זמנית. בפרט, אם אחד מהם נמצא בקטע הקריטיאזי השני יכול להיכנס לקטע הקריטי גם כן, אפילו אם A כבר ממתין להיכנס לקטע קריטי.

על האלגוריתם לקיים את כל הדרישות של פתרון בעיית קטע קריטי, חוץ מהוגנות(FAIRNESS).

(13 נק׳) א. ממשו את הפרוצדורות:

A-enter ,A-exit, B-enter , B-exit, C-enter ו C-exit

המממשות את קטעי הכניסה והיציאה של התהליכונים לקטע קריטי בצורה היעילה ביותר.

בסעיף זה מותר להשתמש אך ורק במשתנים הבאים:

1. סמפורים S1 ו-S2 המאותחלים שניהם לערך 1.
2. משתנה C שמאותחל לערך 0.

אין להשתמש בהמתנה פעילה(busy waiting). כלומר, תהליכונים יכולים להמתין רק ע״י שינה על סמפור.

אין חובה להשתמש בכל המשתנים.

```
A enter
down(S1)
while (C≠0){
    up(S1)
    down(S1)}
down(S2)
```

```
B/C enter
down(S1)
C++
up(S1)
down(S2)
up(S2)
```

```
A exit
up(S2)
up(S1)
```

```
B/C exit
down(S1)
C --
up(S1)
```

(9 נק') ב. גם בסעיף זה עליכם לממש את הפרוצדורות :

A-enter ,A-exit, B-enter , B-exit, C-enter ו C-exit

אבל בסעיף זה מותר להשתמש אך ורק במשתנים הבאים : m1, m2 ו-c שמאותחלים לאפס. התהליכונים הנאלצים להמתין יעשו זאת באמצעות busy waiting בעזרת פקודה test-and-set .

להזכירכם פעולה (test-and-set(c,1 מתבצעת באופן אטומי, כותבת ערך 1 למשתנה c ומחזירה את ערכו הקודם שלו. אין להשתמש בסמפורים בסעיף הזה.

```
A enter
while(test-and-set(c,1)=1);

A exit
c=0;

B enter
while(test-and-set(c,1)=1 OR m2=0);
m1=1;

B exit
m1=0;
if(m2=0)
   c=0;

C exit
m2=0;
if(m1=0)
   c=0;

C enter
while(test-and-set(c,1)=1 OR m1=0);
m2=1
```

17
(1)
8+9

צריך מימוש כמו קוראים כותבים

שאלה 2 (15 נקודות)

(10 נק׳) א. השלימו את תיאורו של האלגוריתם ״קבוצת העבודה״ WS (working set) להחלפת דפים:

כאשר צריכים לפנות דף, עוברים על הדפים שנמצאים בזיכרון ובודקים לכל דף:

אם R=1, אזי עדכן את הזמן הווירטואלי של הדף לזמן הווירטואלי הנוכחי ועדכן את R=0. לדף הייתה התייחסות (כלומר פנו אליו) לאחרונה, ולכן הוא איננו מועמד לפינוי.

אם R=0, הדף מועמד להיות מפונה.

חשב את age = current virtual time – last virtual time

אם τ < age, אזי הדף כבר אינו בקבוצת עבודה ולכן הוא מפונה.

אם τ => age, אזי הדף עדיין נמצא בקבוצת עבודה, אבל ייתכן שיפונה אם בסוף המעבר על כל הדפים יימצא שהוא היה ה ׳ותיק׳ ביותר מבין כל הדפים בקבוצת העבודה עם סיבית התייחסות ״כבויה״. (מובן שפינוי דף כזה שהיה בקבוצת עבודה יתבצע רק אם לא התגלה דף מחוץ לקבוצה).

אם עברנו על כל הדפים ולא מצאנו דף עם סיבית התייחסות ״כבויה״, נבחר באקראי דף עם R=~~0~~ 1 ונפנה אותו. במידת האפשר, הדף הנבחר יהיה עם סיבית שינוי ״כבויה״, כדי לחסוך כתיבה לדיסק.

(5 נק׳) ב. במה אלגוריתם WSClock (working set clock) עדיף על LRU?

תשובה והסבר:

WSClock הרבה יותר ישים מימוש ל- LRU

שצריך קשה למימוש.

בנוסף WSClock מתאים לקבוצת העבודה, שבמקרים

מסוימים מהווה יתרון.

ההסבר כללי ולא עונה על השאלה

10
(2)

נא לא לכתוב בשוליים

שאלה 3 (18 נקודות)

נתונה מערכת קבצים דמוית ext2 מבוססת I – NODES עם 12 מצביעים ישירים, והשאר המצביעים הרגילים. במערכת מטמון מערכת קבצים מבוטל.

גודל הבלוק הוא 1KB, אורך הכתובת 4 BYTE.

(8 נק') א. התוכנית מגרילה בכל פעם באופן רנדומאלי אה המיקום (offset) בקובץ נתון וקוראת 100 בתים של נתונים ממיקום זה בקובץ.

הניחו כי לכל קובץ ממנו קוראת התוכנית מוקצה מקום רציף בדיסק.

לאחר הרצת התכנית מספר רב של פעמים על קבצים שונים שמו לב על תופעה הבאה: משך הזמן הנדרש להרצת התכנית שונה עבור קבצים בגדלים שונים. עבור קבצים קטנים (100KB לכל היותר) משך ההרצה קצר בצורה משמעותית מקבצים גדולים (יותר מ-1GB). כיצד ניתן להסביר תופעה זו בהסתמך על מאפייני המערכת? הסבירו בקצרה אך במדויק.

single-indirect ✓

הסבר:

קבצים קטנים יעשו שימוש רק בגישת ~~singleindirect~~ שדורשת

לקריאת בלוק מצביעים אחד נוסף למידע, קבצים גדולים

יעשו שימוש גם ב triple-indirect שדורש 4 גישות לכל בלוק.

(5 נק') ב. האם התופעה הנזכרת לעיל בהכרח תהיה קיימת במערכת שמחזיקה את הקבצים לפי הגודל שלהם בדרך הבאה:

הקבצים הגדולים בתחילת הדיסק הקשיח והקטנים בסופו. אין צורך להתייחס לשאלה איך היא עושה את זה.

*תפס, חיפשתי כך, לא מצאתי

תשובה והסבר:

בקבצים גדולים עדיין יידרשו יותר גישות לדיסק

~~[illegible]~~ בשביל לקרוא את ~~[illegible]~~ טבלאות

המצביעים, לכן התופעה עדיין קיימת.

יש עניין הזזות ראשים

(5 נק') ג. תוכנית חיפוש של תבניות(רצפים של תווים) בקבצים במערכת הנ"ל מביאה את הקבצים לזיכרון. מהו מספר הבלוקים המקסימאלי שצריך לקרוא מתוך מערכת הקבצים הנ"ל לזיכרון על מנת לבצע חיפוש בקובץ שגודלו 1024 KB?

חישוב והסבר:

בלוקי data

i-node, מכיל 12 מצביעים

אך ראשון ב- double-indirect

1024 + 1 + 1 + 1 + 3 = 1030

single-indirect

מכיל 256 מצביעים ✓

3 טבלאות מצביעים מהבלוק השני ב- double-indirect

15

(3)

חלק ב

עליכם לענות על **חמש** השאלות 4-8.

(5 נקודות לשאלה ; 25 נקודות לחלק זה).

שאלה 4

הסבירו למה במערכת XV6 שתומכת בקונטיינרים בעקבות כתיבת "cpu+" לקובץ cgroup.subtree_control בבדיקה cat cgroup.subtree_control מופיע רק "cpu" בלי "+" ? רמז : מה הוא הסוג של מערכת קבצים cgroup שהקובץ cgroup.subtree_control הוא חלק ממנה.

5
(4)

תשובה והסבר :

המערכת היא pseudo-file-system

שאלה 5

מה יהיה הפלט אחרי הרצת הפקודות הבאות במערכת XV6 שתומכת בקונטיינרים?

ידוע ש PID הגדול ביותר שהיה במערכת לפני הרצת unshare (PID_NS) היה 15 .

ניתן להניח שכל קריאות מערכת מצליחות.

```
unshare(PID_NS);
pid = fork();
if (pid > 0)  printf(stdout, "Grandparent's perspective on the child. PID=%d \n", pid);
else {
   pid = fork();
   if (pid == 0) printf (stdout, "Son PID=%d \n", getpid());
   else printf (stdout, "Parent's perspective on the child. PID=%d \n", pid);
   }
```

סבא מחוץ לקונטיינר, אבא ובן בתוך אותו קונטיינר

תשובה והסבר :

Parent_Pid=2 Son Pid=3 , Grandparent_Pid = 1

unshare בהתחלה יוצר Pid_NS חדש, קריאת Pid הראשונה

מתייחסת כבר ל- Pid_NS החדש. *סדר ההדפסות לא ידוע.

1
(5)

שאלה 6

מה יודפס בעקבות הרצת קטע קוד הבא בתוך מערכת xv6 שתומכת בקונטיינרים, ניתן להניח שכל קריאות מערכת מצליחות:

```
unshare (MOUNT_NS);
int pid = fork();
if (pid == 0) {
  mkdir (“dirS”);
  mount (“internal_fs_a”,”dirS”,0); //mount a preformatted internal_fs_a
  int fd = open (“dirS/file.txt”,O_CREATE|O_RDWR);
  write (fd ,“123456789\n”,10);
  close (fd);
 }
if (pid > 0) sleep (10000); // wait for child to finish
sleep (10000); // both processes will sleep
if( pid == 0) {
   if (open (“dirS/file.txt”,O_RDONLY) < 0) printf (stderr, "son failed to open\n");
   else printf (stderr, "son succeeded\n");
 }
If (pid>0) {
   if (open (“dirS/file.txt”,O_RDONLY) < 0) printf (stderr, "father failed to open\n");
   else printf (stderr, "father succeeded\n");
}
```

תשובה והסבר:

UNSHARE לפני FORK , כולם באותו MOUNT_NS

"son succeeded"; "fater failed" לא בהכרח בסדר

הקובץ dirS/file.txt נוצר אחרי שהתהליך הבן עשה mount

ל- filesystem פנימי.

4

(6)

שאלה 7

לדיסק בעל 40 גלילים(CYLINDERS), כאשר הזרוע נמצאת בגליל 11 (בכיוון תנועה ושירות בקשות כלפי הגליל בעל המספר הגבוה ביותר), מגיעה סדרת בקשות ק/פ לגלילים הבאים (משמאל לימין):

1, 36, 16, 34, 9, 12

איזה מאלגוריתמי תזמון הדיסק הבאים FCFS , SSF , SCAN יתבצע תוך השגת מינימום מרחק חיפוש כולל? הראו את החישובים בבירור.

5

(7)

תשובה:

FCFS, 11, 1,36,16,34,9,12 : 10+35+20+18+25+3 = 111

SSF - 11,12,9,16,1,34,36 : 1+3+7+15+33+2 = 61

SCAN - 11,12,16,34,36,1,9 : 25+35+8 = 68

אלגוריתם SSF

שאלה 8

איזו תכונה חשובה מוסיפה טכנולוגיית וירטואליזציה VT (Virtualization Technology) למעבד ואיך זה משפר את ביצועי וירטואליזציה?

5
(8)

בנוסף לפקודות במצב משתמש ומצב מיוחס, המעבד יודע לזהות סביבות וגישות, כאשר פקודות במצב מיוחס של Guest-OS, ולזמן את ה-hypervisor, במקום שה-hypervisor יזהה זאת בעצמו (פעולה יקרה).

חלק ג (20 נקודות)

ענו על ארבע שאלות רב-ברירה (אמריקאיות). משקל כל שאלה 5 נקודות.

בכל שאלה יש לבחור את התשובה הנכונה ביותר ולהקיף בעיגול את אות התשובה שבחרתם.

שאלה 9

מהי הסיבה העיקרית לשימוש ב-DMA?

5
(9)

א. מאפשר למעבד לרוץ בקצב מהיר יותר.
[✓ ב.] שיפור ביצועי המערכת ע"י הגדלת המקביליות.
ג. הקטנת העומס על הזיכרון הראשי.
ד. מקסום הניצול של שטח הדיסק.

שאלה 10

כאשר מדובר על מבנה מערכת הפעלה לפי מודל שרת-לקוח (client-server model) מהי התכונה אשר מהווה חסרון מובהק של המודל?

5
(10)

א. העדר מבנה כלשהו. המערכת היא אוסף שגרות אשר כל אחת מהן יכולה לקרוא לשגרה אחרת מן האוסף
ב. חוסר אפשרות התאמה למערכות מבוזרות (distributed systems)
[✓ ג.] תקורת (overhead) התקשורת בין רכיבי המערכת.
ד. כל התשובות הקודמות הן נכונות.

המשך הבחינה בעמוד הבא

נא לא לכתוב בשוליים

שאלה 11

מהו החיסרון המובהק של שיטת ה- rendezvous לעומת ה- mailbox ב- message passing?

[✓ א.] שיטת ה-rendezvous קשה יותר למימוש במערכת הפעלה.
ב. שיטת ה-rendezvous פחות גמישה מכיוון שהתהליכים (השולח והנמען) הופכים להיות מסונכרנים (תלויים זה בזה).
ג. בשיטת ה-rendezvous לא ניתן לממש מניעה הדדית בגישה לקטעים קריטיים (critical sections).
ד. שיטת ה-rendezvous גורמת לסחרור (thrashing)

0
(11)

שאלה 12

נתון כי במערכת מסוימת המשתמשת בשיטת הדפדוף (paging) לצורך ניהול הזיכרון:

- גודל הדף הוא 4 K.
- הכתובת הוירטואלית הנה בת 20 סיביות (bits).
- גודל הזיכרון הפיזי הוא 640 K.
- גודל שורה בטבלת הדפים (page table entry) הוא 16 בתים (bytes).

אם טבלת הדפים הנה טבלת הדפים המהופכת (inverted page table), מהו גודל הטבלה?

[✓ א.] 2560 בתים.
ב. 640 בתים.
ג. 2048 בתים.
ד. 1024 בתים.

5
(12)

10

256 שורות

בהצלחה!

נא לא לכתוב בשוליים